Área do administrador

FATECalçados · Programa de fidelidade · como funciona o código da parte da loja (caixa e gestão)

> A área do administrador é a parte da loja: o **caixa (PDV)** e as telas de **gestão**. Ela fica em `/admin` e só a equipe entra. A página inicial não tem link para ela: chega-se digitando o endereço. Não confundir com o admin do Django, que fica na porta 8000.

1. Quem é administrador

A equipe não tem tabela própria. Cada administrador é um usuário comum do Django (`User`) marcado com `is_staff`. Ele é criado pelo terminal:

```
docker compose exec backend python manage.py criar_admin --usuario gerente --nome "Gerente"
```

- **O comando** (`backend/src/management/commands/criar_admin.py`) só lê os argumentos e pede a senha duas vezes sem mostrar na tela, para ela não ficar no histórico do terminal.
- **As regras** ficam em `services.criar_administrador` (`backend/src/services.py`):
  - **Não aceita usuário só com números.** O login do cliente é o CPF, então nomes só com dígitos ficam reservados para clientes e os dois nunca se confundem.
  - **Não sobrescreve quem já existe.** Para trocar a senha de alguém, é preciso passar `--redefinir`.
  - **Não promove cliente a administrador.** Se o usuário pertence a um cliente, o comando recusa.
  - **Valida a senha** com as mesmas regras do cadastro do cliente: mínimo de 8 caracteres, não pode ser só números nem senha comum, nem parecida com o nome de usuário.
  - **Marca `is_staff` e `is_superuser`.** Por isso o mesmo login também entra no admin do Django, em `localhost:8000/admin/`.
- **Senhas guardadas com Argon2id** (`src/hashers.py`): o banco só tem o hash, que não pode ser revertido. Quem esquece a senha cria uma nova com `criar_admin --usuario <nome> --redefinir`.

2. Tela de login e sessão

A tela de entrada é `/admin/entrar` (`frontend/src/pages/inicio/EntrarEquipe.tsx`). Ela tem o **mesmo visual do login do cliente** (`/login`): as duas usam o componente `TelaAcesso` com o fundo da foto de tênis escondida (`FUNDO_SECRETO`), que aparece quando se passa o mouse. Muda só o conteúdo: título "Área da loja", campos Usuário e Senha e o link "É cliente? Entre na sua conta".

```
/admin/entrar ──POST /api/equipe/entrar/──► services.autenticar_equipe ──► login() cria a sessão
```

- **Não há token.** O login cria uma sessão do Django, guardada no cookie `sessionid`.
- `autenticar_equipe` **recusa quem não tem** `is_staff`**.** Um cliente com a senha certa recebe "Usuário ou senha incorretos."
- `GET /api/equipe/` **diz quem está logado**, ou responde 401. Essa chamada também envia o cookie `csrftoken`.

- **Todo POST leva o cabeçalho** `X-CSRFToken` com o valor desse cookie: é a proteção CSRF do Django.
- **Cliente e equipe dividem a mesma sessão.** Entrar como um no navegador desloga o outro. Para testar os dois ao mesmo tempo, use uma janela anônima.
- Depois de entrar, a pessoa volta para a tela que tinha pedido (ou para `/admin`).

3. Backend: como os dados são protegidos

Em `backend/src/views.py`, toda rota da loja passa por um decorador:

```
def _so_equipe(view):
    def protegida(request, *args, **kwargs):
        if not (request.user.is_authenticated and request.user.is_staff):
            return _erro("Entre com um usuário da equipe da loja.", status=401)
        return view(request, *args, **kwargs)
```

Ele protege estas rotas:

Rota	Tela
`/api/dashboard/`	Dashboard
`/api/acoes/`	Ações do dia
`/api/clientes/` (lista e cadastro), `/api/clientes/<cpf>/`, `/api/clientes/<id>/perfil/`	Clientes e PDV
`/api/vendas/` (lista e registro), `/api/vendas/<id>/`	Vendas e PDV
`/api/avaliacoes/`	Avaliações
`/api/estoque/`, `/api/estoque/<sku>/`	Estoque

A única rota aberta é `/api/produtos/`, porque a loja online do cliente também usa o catálogo.

As camadas

Cada camada só chama a de baixo:

- `views.py` lê a entrada e devolve JSON, sem regra de negócio.
- `services.py` é o único lugar que grava no banco: registrar venda, cadastrar cliente, ajustar estoque. Uma regra violada vira `RegraViolada`, que a view devolve como 400 com `{"erro": "..."}`.
- `indicadores.py`, `acoes.py` **e** `consultas.py` só leem: números do dashboard, listas de ações e listagens.
- `regras.py` guarda os números do programa (níveis, bônus, resgate), sem acesso ao banco.

4. Frontend: como as telas são montadas

Em `frontend/src/routes/index.tsx`, tudo que fica sob `/admin` usa o `MainLayout`. As providers são montadas nesta ordem:

```
MainLayout
 └ ToastProvider
     └ EquipeProvider        ← confere a sessão da equipe
         └ CaixaProvider     ← fila de vendas offline, venda em andamento
             └ AcoesProvider ← listas de ações e o que já foi feito
                 └ Sidebar + cabeçalho + <Outlet/> (a tela)
```

EquipeProvider: o porteiro da área

Arquivo `frontend/src/contexts/EquipeProvider.tsx`:

1. Ao abrir qualquer página `/admin/...`, chama `GET /api/equipe/` e mostra "Conferindo o acesso...".
2. Sem sessão, redireciona para `/admin/entrar` e guarda a página pedida em `state.de`. Depois do login, a pessoa volta direto para onde queria.
3. Com sessão, monta o resto. Como o caixa e as ações só montam depois dessa checagem, nenhuma tela chama a API antes do login.
4. O "Sair" da sidebar chama `staffSignOut`, que encerra a sessão no backend.

As telas

Ficam em `frontend/src/pages/<área>/`:

Endereço	O que faz
`/admin/pdv`	Caixa. Busca o cliente pelo CPF, monta o carrinho, escolhe a numeração, resgata pontos e finaliza. Funciona offline: a venda entra numa fila no `localStorage` e é reenviada quando a conexão volta. Não duplica porque o backend reconhece o `id` da venda. Se o reenvio der 401, a venda continua pendente.
`/admin/dashboard`	Indicadores de pontos, vendas e clientes, com o período na URL e exportação para CSV e PDF.
`/admin/clientes`	Lista com CPF mascarado; o perfil de cada cliente abre em `/admin/clientes/:id`.
`/admin/vendas`	Lista com filtros e o detalhe de cada venda.
`/admin/acoes`	Clientes para contatar (sumindo, pontos vencendo, perto de subir de nível), com mensagem pronta.
`/admin/avaliacoes`	Resumo, distribuição das notas e comentários das avaliações dos clientes.
`/admin/estoque`	Grade por numeração com cores de situação (esgotado, acabando, ok); clicar num tamanho edita a contagem.

Outras peças do layout

- A sidebar lê a lista de rotas de `navegacao.ts`.
- No primeiro acesso, o `TourGuiado` apresenta cada item da sidebar; o botão "Rever o tour" reabre.
- Ctrl/⌘ K abre a busca rápida de telas e clientes.

Chamadas à API

Tudo passa por `frontend/src/services/api/`. O `http.ts` cuida do CSRF e dos erros, e há um módulo por área (`equipe.ts`, `vendas.ts`, `estoque.ts`...). Esses módulos traduzem os campos em português da API para os nomes em inglês que os componentes usam.

5. Como testar

- **Testes do backend:** `backend/src/tests/test_equipe.py` confere que cada rota da loja responde 401 sem login, cobre as regras do `criar_admin` e confirma que um cliente não entra pela tela da equipe.

  ```
  docker compose exec backend python manage.py test src.tests.test_equipe
  ```

- **Swagger:** em `localhost:8000/api/docs/`, faça `POST /api/equipe/entrar/` com um usuário da equipe e depois teste qualquer rota da gestão por ali. A lista completa dos endpoints está em `docs/endpoints-da-api.pdf`.
- **No navegador:** abra `localhost:5173/admin`. Sem sessão, ele manda para `/admin/entrar`.